Ingeniería de Software Asistida por IA

Tema 2: Metodología PCA y Ciclo de Vida

Guía de Estudio Detallada para Estudiantes

¿Qué es la Metodología PCA?

La Metodología PCA es un marco de trabajo (*Framework*) diseñado específicamente para mitigar los problemas de amnesia y desorden inherentes a las Inteligencias Artificiales. No es un lenguaje de programación, sino un sistema de **reglas y etiquetas Markdown** que fuerza a la IA a comportarse de manera estructurada, predecible y modular.

Mientras que un desarrollador común le pide favores a ChatGPT esperando lo mejor, un ingeniero que usa PCA orquesta múltiples "agentes" virtuales (Analista, Arquitecto, QA) limitándolos mediante directivas estrictas para garantizar que el código resultante cumpla con los estándares empresariales.

El Enrutador de Contexto: AGENTS.md

El archivo más importante de cualquier proyecto asistido por IA es el `AGENTS.md`. Este archivo actúa como el "Cerebro" de tu repositorio y debe vivir en la carpeta raíz de tu proyecto.

En este documento se definen las "Personas" o roles que la IA puede asumir. Contiene las instrucciones maestras (System Prompts) que dictan cómo debe razonar la IA antes de programar, qué estándares de seguridad debe seguir y cómo debe comunicarse contigo.

La Regla Fundamental: Cuando inicias un nuevo chat con la IA, tu primer mensaje SIEMPRE debe ser proporcionarle el contenido de `AGENTS.md` o pedirle que lo lea. Esto calibra su comportamiento, erradica el estilo conversacional genérico y la convierte instantáneamente en tu ingeniero experto personalizado.

La Memoria a Largo Plazo: Cicatrices de Guerra

Dado que las IAs sufren de Amnesia de Contexto, tienden a repetir los mismos errores lógicos a lo largo del tiempo. Para solucionar esto, introducimos el archivo `cicatrices_guerra.md` (o el uso de la etiqueta `@learn-error`).

Este documento es un registro histórico de los fracasos de la máquina. Cuando la IA comete un error grave (por ejemplo, usar un método obsoleto de una biblioteca de React), tú no solo le pides que lo corrija. Tú la obligas a registrar ese error, la razón por la que falló y la solución arquitectónica en las cicatrices de guerra. En el siguiente Sprint, le pasas ese registro como contexto obligatorio, inmunizando tu proyecto contra ese error específico para siempre.

El Ciclo de Vida de 10 Pasos (PCA)

Para evitar el caos del *Vibe Coding*, el desarrollo con PCA prohíbe generar código de buenas a primeras. Exige un flujo cíclico e inquebrantable de 10 pasos agrupados en 4 fases críticas:

FASE 1: Preparación (Pasos 1-2)

- **1. Select:** Seleccionas qué bloque pequeño del proyecto vas a atacar.
- **2. Context:** Provees a la IA explícitamente los archivos relevantes (Interfaces, Types) y el `AGENTS.md`. Limitas su visión solo a lo que necesita saber para evitar distracciones matemáticas.

FASE 2: Planificación (Pasos 3-5)

- **3. Prompt:** Haces tu petición clara, exigiendo el análisis PCA.
- **4. Plan:** La IA debe devolver un plan arquitectónico sin código ejecutable.
- **5. Audit:** Tú, el humano, lees el plan, verificas la lógica y lo apruebas o corriges.

FASE 3: Mutación (Pasos 6-7)

- **6. Mutate:** Una vez aprobado el plan, autorizas a la IA a generar el código final.
- **7. Integrate:** Pegas el código en tu editor, pero no asumes que funciona. Verificas que los componentes compilen y que la sintaxis esté libre de errores visibles.

FASE 4: Validación (Pasos 8-10)

- **8. QA (Quality Assurance):** Pruebas la funcionalidad manualmente o mediante pruebas unitarias.
- **9. Refactor:** Si el código es feo, difícil de leer o no cumple con el Principio de Responsabilidad Única, obligas a la IA a limpiarlo inmediatamente.
- **10. Commit:** Haces un guardado en Git de esta versión estable antes de iniciar el ciclo nuevamente.

La Prueba Final: Intercambiabilidad de Roles

La metodología PCA no solo audita a la máquina, también audita al humano. En un equipo de ingeniería real, el código no tiene dueños exclusivos; es propiedad del equipo.

La "Intercambiabilidad de Roles" es una política que dictamina que **cualquier miembro de tu equipo debe ser capaz de explicar la trazabilidad y la lógica del código que la IA generó para ti.**

Si tú generaste el módulo de pagos asumiendo el rol de "Desarrollador Backend", y tu compañero "Frontend" no es capaz de leer y entender tu código de pagos, significa que permitiste que la IA escribiera código ilegible (slop). Bajo esta filosofía, la calidad del código se mide por la facilidad con la que otra persona puede auditarlo.